Inverse Probability Weighting in R

David M. Vock

Inverse Probability Weighting in MSM

We can use IPW weights to estimate the parameters of marginal structural models. Among other things, this helps us control for confounding.

This amounts to fitting a model where the outcome of interest is regressed on the exposure of interest, with each observation weighted by the inverse of the probability of the observed exposure level given the observed value of the confounders.

These slides follow the examples given by the authors of the `ipw` package (van der Wal and Geskus, 2011).

```
library(ipw)
```

Point Treatments: Data

We'll simulate data from 1,000 individuals. Each individual has binary exposure A, continuous response Y, and continuous confounder L such that:

- $L \sim N(10, 5)$
- $logit(P(A = 1)) = L - 10$
- $Y = 10A + 0.5L + N(-10, 5)$

```
set.seed(1738)
N <- 1000
data <- data.frame(l = rnorm(N,10,5))
a.logit <- data$l - 10
prob.a <- exp(a.logit) / (1 + exp(a.logit))
data$a <- rbinom(N, 1, prob.a)
data$y <- 10*data$a + 0.5*data$l + rnorm(N,-10,5)
```

The ipwpoint() function

We want to estimate the stabilized weights:

$$sw_i = \frac{P(A_i = a_i)}{P(A_i = a_i | L_i = l_i)}$$

```
w1 <- ipwpoint(exposure = a, family = "binomial", link = "logit",
                numerator = ~ 1, denominator = ~ l, data = data)
```

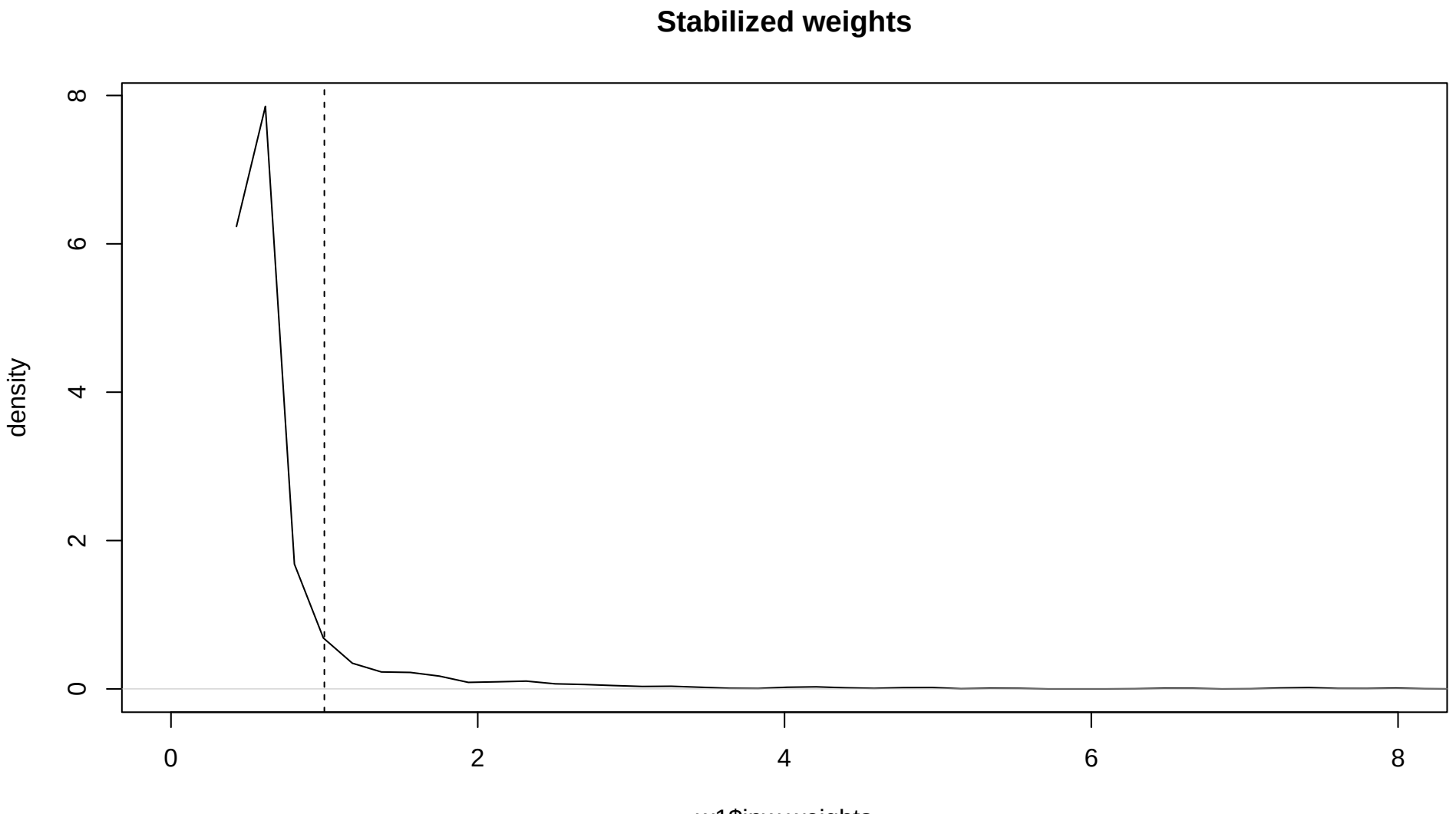
The `ipwpoint()` function is very handy. We must specify

- an `exposure`
- model specifications `family` and `link`
- model formulas `numerator` and `denominator`
- optional truncation factor `trunc`

Plotting weights

We can plot the weights:

```
ipwplot(weights = w1$ipw.weights, logscale = FALSE,
        main = "Stabilized weights", xlim = c(0, 8))
```



Numerator and denominator models

We can also look at the models used in the numerator and denominator:

```
summary(w1$num.mod)
summary(w1$den.mod)
```

Putting it all together

```
library(survey)
data$w <- w1$ipw.weights
msm <- svyglm(y ~ a, design = svydesign(~ 1, weights = ~ w, data = data))
coef(msm)
```

```
(Intercept)           a
  -5.544898   12.166504
```

```
confint(msm)
```

```
                2.5 %    97.5 %
(Intercept) -6.946327 -4.143469
a           10.658432 13.674576
```

Time-varying exposures: data

Now we'll illustrate the use of the `ipw` package for time-varying exposures. The `haartdat` dataset is a simulated time series dataset.

```
data("haartdat")
haartdat[1:10,]
```

```
   patient tstart fuptime haartind event sex age cd4.sqrt endtime dropout
1        1   -100       0        0     0   1  22 23.83275    2900       0
2        1      0     100        0     0   1  22 25.59297    2900       0
3        1    100     200        0     0   1  22 23.47339    2900       0
4        1    200     300        0     0   1  22 24.16609    2900       0
5        1    300     400        0     0   1  22 23.23790    2900       0
6        1    400     500        0     0   1  22 24.85961    2900       0
7        1    500     600        0     0   1  22 25.94224    2900       0
8        1    600     700        1     0   1  22 26.03843    2900       0
9        1    700     800        1     0   1  22 26.72078    2900       0
10       1    800     900        1     0   1  22 27.47726    2900       0
```

We'll want to adjust for the confounding CD4 counts, which vary over time. The treatment `haartind` is also time-varying.

The ipwtm() function

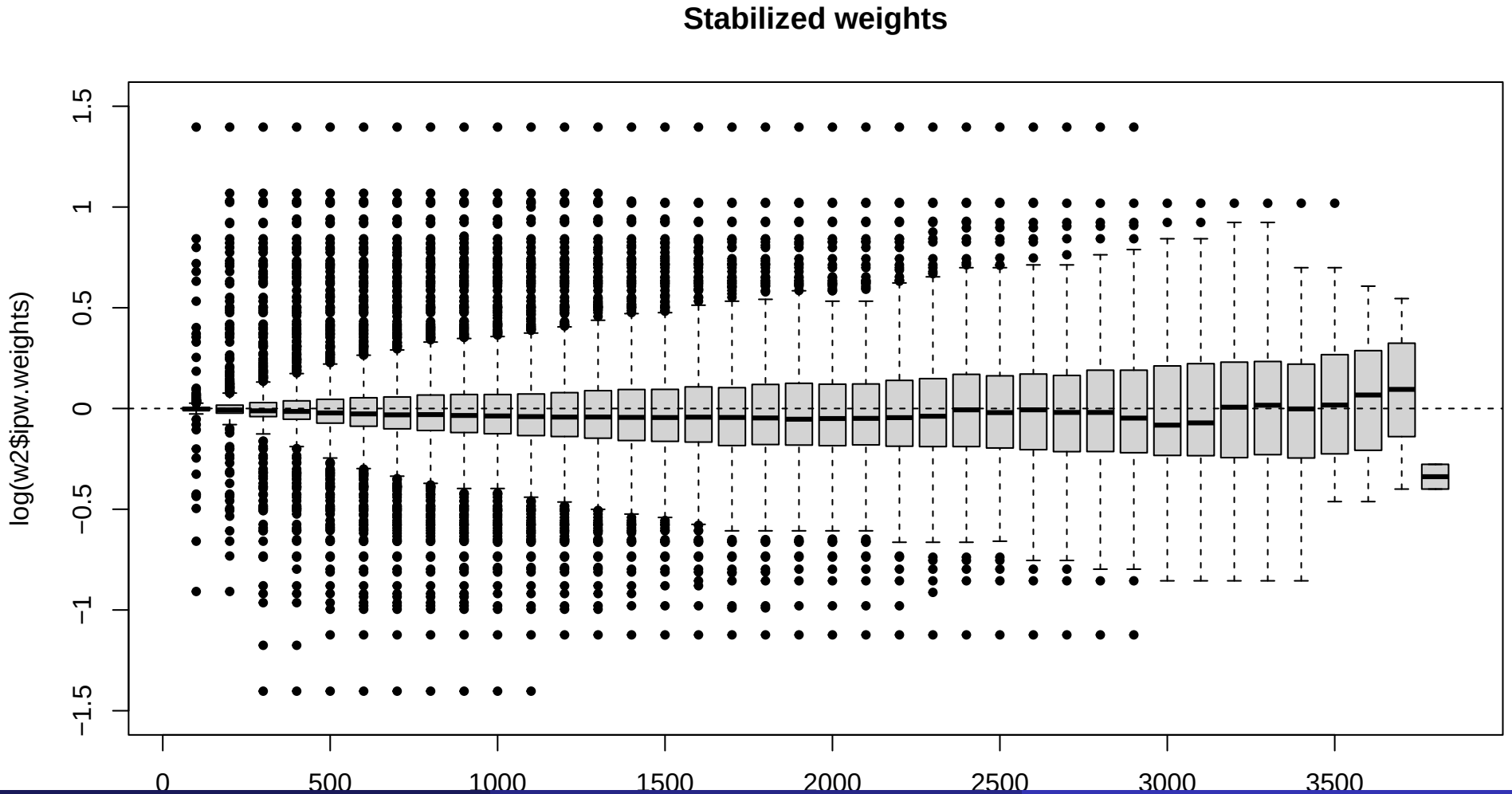
We use the `ipwtm` function to estimate weights. We adjust for sex and age in both the numerator and denominator while adjusting for CD4 count in only the denominator. The `type` option tells R that we only want to estimate weights up until exposure begins.

$$sw_{ij} = \prod_{k=0}^{j} \frac{P(H_{ik} = h_{ik} | \bar{H}_{ik-1} = \bar{h}_{ik-1}, \mathbf{V}_i = \mathbf{v}_i)}{P(H_{ik} = h_{ik} | \bar{H}_{ik-1} = \bar{h}_{ik-1}, \bar{L}_{ik-1} = \bar{l}_{ik-1}, \mathbf{V}_i = \mathbf{v}_i)}$$

```
w2 <- ipwtm(exposure = haartind, family = "survival",
              numerator = ~ sex + age,
              denominator = ~ cd4.sqrt + sex + age,
              id = patient, tstart = tstart, timevar = fuptime, type = "first",
              data = haartdat)
```

Plotting the weights

```
ipwplot(weights = w2$ipw.weights, timevar = haartdat$fuptime,
        binwidth = 100, ylim = c(-1.5, 1.5), main = "Stabilized weights",
        xaxt = "n", yaxt = "n")
axis(side = 1, at = c(0, 5, 10, 15, 20, 25, 30, 35),
     labels = as.character(c(0, 5, 10, 15, 20, 25, 30, 35)*100))
axis(side = 2, at = c(-1.5, -1, -0.5, 0, 0.5, 1, 1.5),
     labels = as.character(c(-1.5, -1, -0.5, 0, 0.5, 1, 1.5)))
```



Putting it all together

```
summary(coxph(Surv(tstart, fuptime, event) ~ haartind + cluster(patient),
              data = haartdat, weights = w2$ipw.weights))
```

```
Call:
coxph(formula = Surv(tstart, fuptime, event) ~ haartind, data = haartdat,
    weights = w2$ipw.weights, cluster = patient)

  n= 19175, number of events= 31

            coef exp(coef) se(coef) robust se      z Pr(>|z|)
haartind -0.9921    0.3708   0.4491    0.4613 -2.151   0.0315 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

         exp(coef) exp(-coef) lower .95 upper .95
haartind    0.3708      2.697    0.1501    0.9158

Concordance= 0.597  (se = 0.03 )
Likelihood ratio test= 5.81  on 1 df,   p=0.02
Wald test            = 4.62  on 1 df,   p=0.03
Score (logrank) test = 5.26  on 1 df,   p=0.02,   Robust = 5.43  p=0.02

  (Note: the likelihood ratio and score tests assume independence of
     observations within a cluster, the Wald and robust score tests do not).
```